

State and Action Formulation — SIH26055 Scan Scheduler

Version 1 · 2026-09-06 · Owner: Aamir. Audience: the RL lane, as the input to the reward / PPO work.

Read this before designing anything against it. This document is **not a proposal**. The state and action spaces described here are **already built, tested and running** — they are what `rfenv/env.py` has exposed since 2026-09-04, what all seven baseline schedulers were measured on over 1,539 episodes, and what the four validation gates ran against. Every choice below is a recorded decision in `docs/project/DECISIONS.md` with its evidence attached.

This is a **write-up of an existing interface**, in the requested format, so the RL lane does not have to re-derive it — and so nobody accidentally designs against a different one.

Verification stamp. Every runtime figure in this document was re-checked on **2026-09-06** by importing `rfenv` and printing the spaces and constants (the command is in §6). `pytest tests -q` → **229 passed**. Numbers attributed to a decision entry (D3, D30, D31) are quoted from that entry's own recorded measurement and are labelled as such, not re-measured here.

0. The one-line answer

Current situation → What the agent knows (STATE) → What it can do (ACTION)

t slots elapsed of 600 (30 s); 36 bands, unknown who is transmitting	109 floats in [0,1], built ONLY from its own past looks: per-band hit rate, visit density, staleness, + clock	pick 1 of 36 bands to point the receiver at next. That is the entire action space.
---	--	---

The problem statement calls interception *"a two dimensional search problem since it involves adjusting receiver's frequency at correct time"* (SIH26055_PROBLEM_STATEMENT.md §Detailed Description). Those two dimensions — **which frequency**, **at what time** — are exactly the action space below. There is no third dimension, and §3.4 explains why.

The PS's primary objective is "*to minimize intercept time and ensure a high interception rate*". Both state and action are justified below against those two objectives specifically.

1. The current situation — the decision the agent faces

At each step the receiver is idle and must be pointed somewhere. Its world:

- **36 frequency bands**, centres 250 MHz $\rightarrow$ 17,750 MHz on 500 MHz spacing (`rfenv/constants.py:20`, read from `metadata/receiver/dwell_centres_mhz`; verified identical across all 47 train scan configs). Half-width is ± 500 MHz, so **adjacent bands overlap by half** — D3's measurement: 99.985% of 4,393,233 train scan pulses fall within ± 500 MHz of the dwell centre active at their ToA, against 50.92% for a disjoint ± 250 tiling.
- **A 30 s episode divided into 600 slots of 50 ms** (`rfenv/constants.py:43-45`). 30.0 s is Turing's own `collection_time_s`, identical across all 47 train pairs.
- **A binary world status per (band, slot)** — the PS's own framing: *"The status of environment for each frequency band at each time step can be recorded as a transmission or a non-transmission."*

- **No prior knowledge whatsoever.** Every episode starts cold — no threat library, no carry-over, no prior on who transmits where (**D20**). This is the PS's "*absence of prior reliable intelligence*" made literal, and it is what keeps the explore/exploit tension real: with a prior, camping on the busiest band would be defensible.

Looking at one band means **not** looking at the other 35, and an emitter that transmits while the receiver is elsewhere is missed. Against **spatially scanning and frequency agile** emitters, a band that was empty a second ago may be busy now. That is the whole problem.

2. STATE — what the agent knows at each timestep

2.1 The observation space

```
observation_space = spaces.Box(low=0.0, high=1.0, shape=(109,), dtype=np.float32) # rfenv/env.py:162
```

109 = 36 × 3 + 1. Every component is natively a fraction, so the box is the unit interval and **no scaling or normalisation layer is needed anywhere** — verified at runtime, `obs.min() = 0.0`, `obs.max() = 1.0`. Ratified as **D34** (SETTLED).

Layout, in order (`rfenv/env.py::_observation`):

Index	Component	Exact definition	Type / range
0:36	per-band hit rate	<code>hits[b] / slots_looked[b]</code> ; <code>0.0</code> if band never looked at	float32, [0, 1]
36:72	per-band visit density	<code>slots_looked[b] / max(t, 1)</code> — fraction of spent airtime given to band <i>b</i> . Sums to exactly 1 across bands once <code>t ≥ 1</code> ; all-zero in the <code>reset()</code> observation	float32, [0, 1]
72:108	per-band staleness	<code>(t - last_slot[b]) / 600</code> ; 1.0 if never visited	float32, [0, 1]
108	normalised episode time	<code>t / 600</code>	float32, [0, 1]

Never-visited bands read staleness **1.0** — maximally stale (verified: at `reset()`, all 36 staleness components are exactly 1.0). So an unexplored band looks at least as attractive as one last seen at $t = 0$. That is a deliberate exploration prior baked into the encoding.

2.2 Each state variable against the five criteria

Variable	Why it is relevant	Available / observable in our env?	Agent-controllable?	Type / range	Connection to the PS objectives
Hit rate (36)	What a scheduler needs to estimate detection probability for a band — how likely a look here pays off. It is the exploit signal	Yes , computed from the agent's own dwell outcomes only	Indirectly — the agent moves it by choosing where to look	float32 [0, 1], one per band	PS figure of merit <i>probability of detection</i> . And directly PS: " <i>The model should then be trained based on hits and misses</i> " — this is literally that
Visit density (36)	How the agent's airtime is being spent. Airtime is the only currency in this problem (§3.2), so this is the agent's own budget, made legible	Yes , from own scan history	Yes — it is a direct summary of its own past actions	float32 [0, 1], sums to 1 for $\sum_t \geq 1$ (0 at reset)	PS figure of merit Avg <i>intercept rate</i> ; supports high interception rate
Staleness (36)	What makes this restless rather than a plain bandit. A band ignored for 10 s may have become busy without telling you. The payoff distribution moves while you are not looking. It is the explore signal	Yes , from own scan history	Yes — looking at a band resets its staleness to 0	float32 [0, 1]	PS: scheduling " <i>against spatially scanning and frequency agile emitters</i> "; drives minimise intercept time
Normalised time (1)	Lets the policy behave differently early (explore) and late (exploit). The episode is finite and terminates — there is no state past slot 600	Yes , trivially	No — the clock advances regardless of what it picks	float32 [0, 1]	Both objectives; intercept time is measured against a fixed 30 s horizon

All four are built from the agent's own scan history and nothing else. No prior emitter intelligence enters (D19, D20), and nothing truth-side leaks into the policy's input path (D29).

2.3 The hard constraint the RL lane must not break

This is **D29**, and it is the single most important rule for this lane:

The reward may read truth. The observation may not.

Training is offline and the policy is frozen before deployment, so the reward is a **training-time construct discarded at inference**. It may read the truth grid `Z`, per-emitter signal levels, `first_e` — anything. The **observation** may not, because the 109-vector is all a fielded receiver would actually have.

This is enforced in code, not by convention. `rfenv/baselines.py:69`:

```
OBSERVABLE_INFO = frozenset({"slot", "time_s", "band", "dwell_slots", "Y"})
```

`baselines.guarded()` strips `info` down to that set before any policy sees it, so a policy reaching for `Z`, `pulses` or `first_intercept` raises a `KeyError` **instead of quietly scoring well**. `baselines.make()` wraps every deployable rung in `guarded()` automatically.

RL lane: wrap your policy in `guarded()` too. A `KeyError` catching an information leak is far better than a suspiciously good score catching it.

2.4 What the state deliberately excludes, and why

Excluded	Why
Pulse count, peak amplitude within the dwell	Available at L2 and richer than the PS's framing, but the PS states the environment status as binary — <i>"a transmission or a non-transmission"</i> (D34)
Anything per-emitter	D19 : any design needing per-emitter attribution from observations inherits a deinterleaving problem . Avoid needing it. Deinterleaving is explicitly not the deliverable (D12)
Truth grid <code>Z</code> , <code>first_e</code> , per-emitter levels	D29 — reward-side and evaluator-side only
Any prior threat library	D20 — cold start every episode
AoA and PulseWidth	D30 — still open, and owned by the RL lane. See §4.

3. ACTIONS — what the agent can actually decide

3.1 The action space

```
action_space = spaces.Discrete(36) # rfenv/env.py:159
```

Criterion	Answer
What it is	"Tune the receiver to band <i>a</i> and dwell there for that band's native dwell time"

Criterion	Answer
Why relevant	It is the <i>entire</i> lever the receiver has. The PS's two search dimensions are frequency and time; choosing a band now is choosing both
Available in our env?	Yes — implemented, and all seven baseline schedulers drive the env through it
Agent-controllable?	Yes. This is the only thing the agent controls (see §3.4)
Type / range	Discrete integer, 0 ... 35, one per frequency band
Legality	All 36 actions are legal at every timestep — no masking layer is needed anywhere (D35)
PS connection	Directly serves both objectives: where you point determines both whether you intercept at all (interception rate) and how soon (intercept time)

3.2 An action's cost is not uniform — this is load-bearing

A band's **dwelt length is a property of the band, not a separate choice**. Seven bands — **0, 1, 6, 7, 17, 18, 19** — carry a 100 ms dwell and consume **2 slots**; the other **29** carry 50 ms and consume **1 slot** (`rfenv/constants.py:33-37, 51`, read from `metadata/receiver/dwell_times_s`; re-verified at runtime 2026-09-06). Turing's full sweep of all 36 is **2.15 s**.

Three consequences the RL lane must not get wrong:

1. **An episode is 600 slots but 300–600 `step()` calls (D35)**. Camping on band 0 (wide) takes 300 steps; camping on band 5 (narrow) takes 600. Both cover exactly 30 s. So anything computed *per step* — advantage normalisation, entropy schedules, "average reward per step" — is measuring something different from what the metrics measure.
2. **Airtime is the only currency**. Retuning is free — D31's measurement puts it under 1 μ s, i.e. under 0.002% of a slot. A wide band genuinely costs twice as much of the episode as a narrow one.
3. **Therefore reward is per slot, and a dwell's reward is the sum of its slots' (D31)**. A wide band can earn up to +2. The naive "+1 per dwell" is **wrong** and was explicitly rejected: it would make those seven bands strictly dominated — and per D31 they are the bands that matter most, carrying a recorded mean of 764.6 emitter-frequency placements against 170.4, and the slowest rotators (bands 0 and 1 at median 3.00 rpm).

3.3 An overrunning dwell is clipped, not made illegal

A wide-band action at slot 599 is clipped to one slot rather than forbidden (D35), so the action space never changes shape mid-episode and no policy needs a masking layer for one edge case.

3.4 What is deliberately NOT an action, and why

This is the part most likely to be read as something missing. The agent does **not** choose any of the following, and each has a recorded reason:

Not an action	Why not
Dwell length / bandwidth	Fixed per band by Turing's own receiver configuration, adopted unchanged (D3). Reproducing that configuration exactly is what makes our numbers comparable to their recordings
Detection threshold γ	$\gamma = -111$ dBm is a frozen receiver property , not an agent knob (D15, D21, D42). It is calibrated and swept as an experiment, never learned. No reward and no policy can move P_d or P_{fa}
Antenna steering / look direction	This receiver tunes; it does not steer . The PS frames the search as frequency $\times$ time only, so AoA can never be something a dwell is spent on (D30)
Number of bands / band geometry	Frozen (D42), enforced by <code>tests/test_freeze.py</code> with per-value literals plus a SHA-256 digest. Changing it forces re-validation from gate 1 and a re-run of every baseline (D25)

So: "what controllable parameters does the agent have?" $\rightarrow$ **exactly one, which band to look at next**. The richness of this problem is not in the action space. It is in *when* and *in what order*, across 300–600 sequential decisions taken with no prior.

4. The one genuinely open state question: D30

D30 (OPEN) — do AoA and PulseWidth enter the observation?

DECISIONS.md marks this "**owned by the RL lane**"; RL_LANE_HANDOFF.md §5 assigns it to role **R3**. It is the only unanswered state-space question, and it is deliberately unanswered.

The case for. To an agent seeing only binary hit/miss, a dwell that finds a **new** emitter and one that re-finds a **known** emitter are indistinguishable. D30's recorded walk found that under a camper policy $\sim 99\%$ of dwells are redundant and *every one of them looks like a hit*. AoA separates them: assigning each pulse to the emitter with the nearest median bearing scored 96.7% (`config_2`), 99.3% (`config_59`) and 86.1% (`config_921`) against true labels.

The case against. That accuracy is a **ceiling**, not an achievable figure — it used true labels and whole-episode medians. It degrades exactly where the problem is hardest (96.7% $\rightarrow$ 86.1% going from 19 to 99 emitters). And it costs real structure: L1's grid combines cells by max, which is meaningless on per-emitter bearings, and the fixed-width 109-vector would need a variable-length bearing set encoded by binning or online clustering.

The trigger that decides it: a trained agent failing to explore in a way that hit rate, visit density and staleness demonstrably cannot fix. Measurable **once a policy exists**, meaningless before. `PulseWidth` rides on the same decision.

Direct consequence for the reward lane — read this before choosing a reward. One of the three reward candidates is `first_intercept` (+1 per first intercept of an emitter, D29 candidate 3). Novelty is a **truth-side** quantity: the reward can see it, the observation cannot. Choosing that reward therefore trains the agent to value something it **provably cannot perceive at inference** — it would have to approximate novelty with staleness, which measures *time since looked*, not *have I already found everyone here*. `hit_z` and `hit_y` do not have this problem. If the reward lane lands on `first_intercept`, **D30 stops being optional** — AoA is the observable that would make it actionable rather than merely scorable.

Why it is safe to leave open: the observation vector is **explicitly not on the freeze list** and the four validation gates never read it. Adding AoA later costs a retrain of the policy's input layer — **not** a re-validation of the environment, and **not** a re-run of the baselines.

5. Frozen vs. open — what you may and may not move

Frozen (D42), enforced by `tests/test_freeze.py`: band geometry, slot clock, dwell schedule, N , σ , γ , `PD_POPULATION`. **The action space follows directly from these and does not move.** Editing `rfenv/constants.py` turns the suite red, and by **D25** any change forces re-validation from gate 1 plus a re-run of every baseline.

Deliberately not frozen — the RL lane's to move: - the **reward** — three candidates already written and selectable by name: `hit_z`, `hit_y`, `first_intercept` (`rfenv/env.py:113`, verified at runtime). The selection rule is **D47**; the reward/observation asymmetry is **D29**; per-slot summation is **D31** - **observation extensions** — **D30** (§4) - the per-episode scenario draw

6. Verify any of this yourself

```
.venv/bin/python -m pytest tests -q          # 229 passed, re-run 2026-09-06

.venv/bin/python -c "
from rfenv.env import ScanEnv, REWARDS
from rfenv.scenario import EmitterPool
from rfenv.baselines import OBSERVABLE_INFO
env = ScanEnv(pool=EmitterPool.from_train(), reward='hit_z')
obs, info = env.reset(seed=0)
print('action_space      ', env.action_space)          # Discrete(36)
print('observation_space', env.observation_space)      # Box(0.0, 1.0, (109,), float32)
print('obs shape/range  ', obs.shape, obs.min(), obs.max())
print('rewards available', sorted(REWARDS))
print('observable info  ', sorted(OBSERVABLE_INFO))
"
```

Where the reasoning lives. The agent interface is documented in the module docstring at the top of `rfenv/env.py`. The decisions are in `docs/project/DECISIONS.md` — **D3, D12, D15, D16, D19, D20, D21, D25, D28, D29, D30, D31, D34, D35, D42, D47**. The buildable spec is `docs/project/ENVIRONMENT_SPEC.md` §L3.

Do not redefine a metric here or anywhere else. `docs/project/EVALUATION.md` is the single authority on metrics, baselines, gates and protocol — including the fact that the bar for RL is **rung 5** (interception ratio 0.1104, censored intercept time 3.20 s), not round-robin.